Intro to LLMs

with R and ellmer

Hadley Wickham

Chief Scientist, Posit

Get started by following the setup instructions at https://github.com/hadley/workshop-llm-hackathon

CC BY SA

Framing

- Using LLMs as a data scientist, from a programming language (R).
- Practical, actionable, non-nonsense, minimal-jargon
- We’ll treat LLMs as black boxes, focussing on what they can do, not how they work.
- Just enough knowledge so you know what to search for (or ask an LLM about!)

1. Set up
2. System prompt
3. Model choice
4. Tokens
5. Anatomy of a conversation

Setup

Getting started



- You'll need ellmer: `pak::pak("ellmer")`
- And an account with an LLM provider. I strongly recommend starting with Claude, OpenAI, or Gemini. (Gemini is slightly weaker but is free.)
- If you already pay for Claude, ChatGPT etc, unfortunately you still need a separate account for API access.
- You can be successful with local models but it's much harder (and slower) and I don't recommend it when getting started.

Your turn: get this working

https://github.com/hadley/workshop-llm-hackathon

```
library(ellmer)


# Pick one
chat ← chat_claude("Be terse")
chat ← chat_openai("Be terse")
chat ← chat_google_gemini("Be terse")

chat$chat("What is the capital of France?")
```

Your turn: answer these questions

https://github.com/hadley/workshop-llm-hackathon

- What does the first argument to chat_*() do? What happens if you remove it? How does it affect the responses?
- What does live_console(chat) do? What about live_browser(chat)? When you exit them does the chat remember the conversation? How could you tell?
- How much money have you spent so far?
- If you create two different chat objects do they know about each other?
- Can you send a plot to the model? Can you send an image? A pdf?

Initial vocabulary

Provider

System prompt

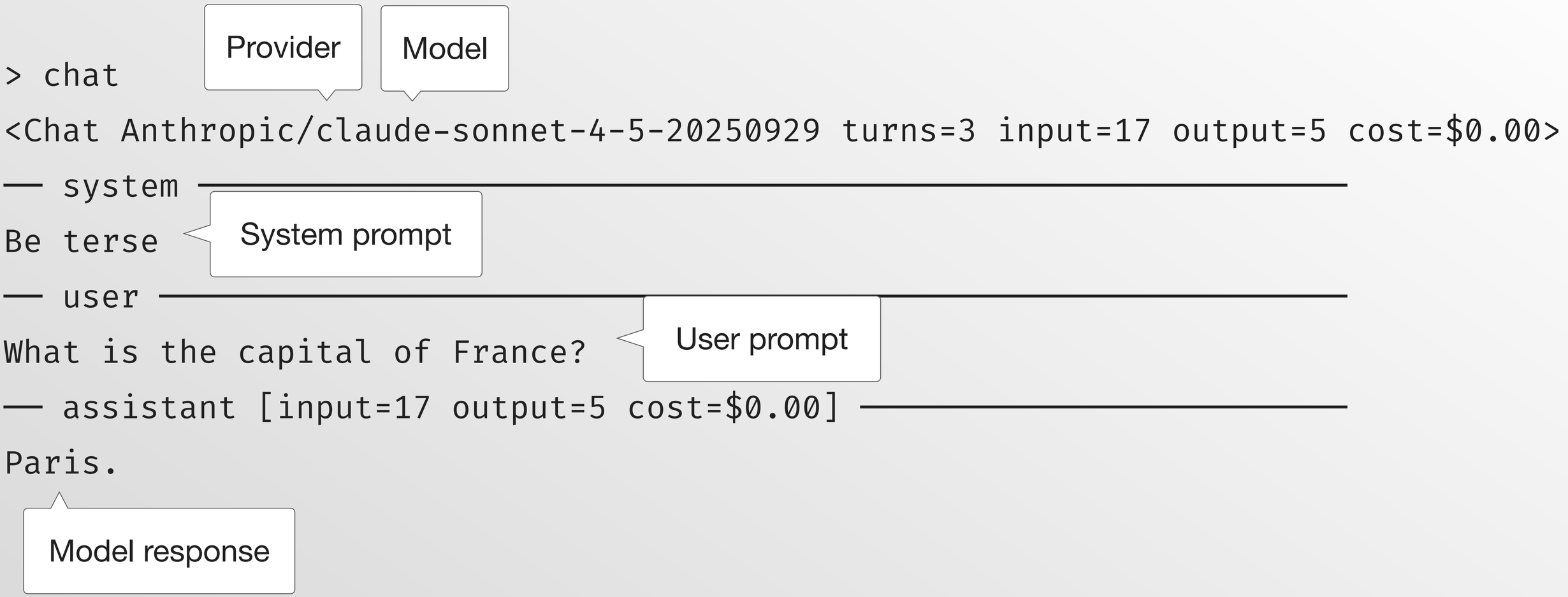
```
chat ← chat_claude("You are a terse assistant.")
#> Using model = "claude-sonnet-4-5-20250929"
```

Model

```
chat$chat("What is the capital of France?")
```

User prompt

Initial vocabulary



User and model responses always come in pairs

System prompt



LLM



LLM with system prompt

System prompt sets the baseline for responses

```
chat ← chat_claude()
chat$chat("What is the capital of the moon?")

chat ← chat_claude("Answer with a Afrikaans accent")
chat$chat("What is the capital of the moon?")

chat ← chat_claude("
  You're an alien that lives on the moon. Really commit to the bit"
)
chat$chat("What is the capital of the moon?")
```

Why use a system prompt?

```
# You could do this:
chat ← chat_claude()
chat$chat("
  You're an alien that lives on the moon. Really commit to the bit"
)
chat$chat("What is the capital of the moon?")


# But:
# (a) you get an extra round of output you don't care about
# (b) the model may forget those instructions if the conversations
#     gets long
# (c) the user (if not you) can override
```

Your turn

```
# Write a system prompt to get output closer to what you
# would personally prefer

chat ← chat_claude()
chat$chat("How can I compute the mean and median of variables a, b, c,
and so on, all the way up to z, grouped by age and sex.")
```

Model choice

Model determines cost/(quality + speed) trade-off

	Bigger	Regular	Smaller	Smallest
Claude	claude-opus-4-1-20250805	claude-sonnet-4-5-20250929	claude-haiku-4-5-20251001	
Gemini		gemini-2.5-pro	gemini-2.5-flash	gemini-2.5-flash-lite
OpenAI	gpt-5-pro	gpt-5.1	gpt-5-mini	gpt-5-nano

Model names & versions are a confusing mess

	Bigger	Regular	Smaller	Smallest
Claude	claude-opus-4-1-20250805	claude-sonnet-4-5-20250929	claude-haiku-4-5-20251001	
Gemini		gemini-2.5-pro *gemini-3-pro-preview*	gemini-2.5-flash	gemini-2.5-flash-lite
OpenAI	gpt-5-pro	gpt-5.1	gpt-5-mini	gpt-5-nano

So ellmer picks a default for you

	Bigger	Regular	Smaller	Smallest
Claude	claude-opus-4-1-20250805	claude-sonnet-4-5-20250929	claude-haiku-4-5-20251001	
Gemini		gemini-2.5-pro	gemini-2.5-flash *oops*	gemini-2.5-flash-lite
OpenAI	gpt-5-pro	**gpt-4.1**	gpt-5-mini	gpt-5-nano

But you can always pick your own

```
chat_claude(model = "claude-haiku-4-5-20251001")
chat_google_gemini(model = "gemini-2.5-flash-lite")
chat_openai(model = "gpt-5.1-nano")

models_claude()
models_google_gemini()
models_openai()
```

Models are priced according to input and output tokens

	Bigger	Regular	Smaller	Smallest
Claude	$15 / $75	$3 / $6	$1 / $5	
Gemini		$1.25 / $10	$0.30 / $1.50	$0.10 / 0.40
OpenAI	$15 / $120	$1.25 / $10	$0.25 / $2	$0.05 / $0.40

What’s a token?

What is a token?

- Featuring engineering to maps words to numbers.
- Common words represented with a single number
  - What is the capital of Paris?
  - 4827, 382, 290, 9029, 328, 12650, 30
- Other words may require multiple numbers:
  - counterrevolutionary
  - 32128 (“counter”), 264 (“re”), 9477 (“volution”), 815 (“ary”)
- See details at https://tiktokenizer.vercel.app/

Why do tokens matter?

- APIs are priced by input and output tokens
- Context size
  - 1M for claude sonnet + gemini flash, 400K for gpt-5
  - 200K = 150,000 words / 300-600 pages / 1.5-2 novels

You see the price when you print the chat

```
> chat
<Chat Anthropic/claude-sonnet-4-5-20250929 turns=3 input=17 output=5 cost=$0.00>
── system ──────────────────────────────────────────────────────────────────────
Be terse
── user ────────────────────────────────────────────────────────────────────────
What is the capital of France?
── assistant [input=17 output=5 cost=$0.00] ────────────────────────────────────
Paris.
```

ellmer does its best to compute prices accurately but it has to compute from first principles (token counts) and there are always small wrinkles.

Or call $get_tokens() / token_usage()

```
> chat$get_tokens()
# A tibble: 2 × 5
  input output cached_input cost       input_preview
  <dbl>  <dbl>        <dbl> <ellmr_dl> <chr>
1    25    320            0 $0.00      "Text[\n  You're an alien that lives …
2   356    336            0 $0.01      "Text[What is the capital of the moon…

> token_usage()
      provider                      model input output cached_input price
1    Anthropic claude-sonnet-4-5-20250929   560   1529            0 $0.02
2       OpenAI                     gpt-4.1    36      5            0 $0.00
3 Google/Gemini            gemini-2.5-flash    22    697            0 $0.00
```

One row for each for **pair** of turns

Caching

- What about cached_input?
- Claude, Gemini, and OpenAI all offer some form of caching, so that if you reuse the same part of a conversation you can save money.
- This is automatic and straightforward for Gemini and OpenAI: prompts are cached automatically, and cached input costs 10% of usual price.
- It's more complicated for Claude, because you'll pay 125% when caching. On the whole this should still save you money so it's enabled by default.
- Will only kick in after some threshold (>1-4k)

NB: tokens accumulate over the course of a conversation

```
chat ← chat_claude()
chat$chat("Tell me a story about a statistician visiting South Africa")
chat$get_tokens()

chat$chat("
  Who were the main characters in this story?
  Answer with a bulleted list
")
chat$get_tokens()

chat$chat("What's the moral of the story?")
chat$get_tokens()
```

How does caching affect price? Experiment

```
news ← readLines(file.path(R.home("doc"), "NEWS"))

# I recommend trying a cheaper model here.
# Typically still good at summarisation tasks
chat ← chat_claude(
  system_prompt = c("Answer questions about the R News file included below", news),
  model = "claude-haiku-4-5-20251001"
)

chat$chat("Summarise changes to sort")
chat$get_tokens()
chat$chat("Summarise changes to order")
chat$get_tokens()
```

Anatomy of a conversation

Overview

- Each interaction is a **pair** of user and assistant **turns**, corresponding to a HTTP request and response.
- The API is entirely stateless, despite conversations being very stateful!
- Can see exactly what ellmer with an httr2 option, which we'll use to explore what's going on under the hood.

HTTP request

```
{
    "model": "claude-3-7-sonnet",
    "system": "You are a terse assistant.",
    "messages": [
        {
            "role": "user",
            "content": [
                {
                    "type": "text",
                    "text": "What is the capital of France?"
                }
            ]
        }
    ],
    "stream": false,
    "max_tokens": 4096
}
```

Model

System prompt

User prompt

HTTP response

```
{
  ...
  "type": "message",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "The capital of France is Paris."
    }
  ],
  "usage": {
    "input_tokens": 22,
    "cache_creation_input_tokens": 0,
    "cache_read_input_tokens": 0,
    "output_tokens": 18
  }
}
```

Model response

HTTP response


```
{
  ...
  "type": "message",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "The capital of France is Paris."
    }
  ],
  "usage": {
    "input_tokens": 22,
    "cache_creation_input_tokens": 0,
    "cache_read_input_tokens": 0,
    "output_tokens": 18
  }
}
```

Your turn

```
options(httr2_verbosity = 2)

chat ← chat_claude("You are a terse assistant.", echo = FALSE)
chat$chat("Tell me a joke about a statistician and a data scientist")
chat$chat("Explain why that's funny")
chat$chat("Make the joke funnier")

options(httr2_verbosity = NULL)
# Look at the request and response and verify that you see it
# growing. Does this help you understand any features of LLMs
# that you've observed in your own usage?
```